Instructor Handout 5: Process Concept and the Process Control Block

Lecture 5 — What Is a Running Program?

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

October 29, 2026

Instructor Copy — Not for Student Distribution

Concepts That Click

5.1 Before You Teach This Lecture

- **Silberschatz, Galvin & Gagne, *Operating System Concepts*, 8e** — Ch. 3 §3.1: the process (p. 101), process state (p. 103), the process control block (p. 103), threads (pp. 104–105). Primary anchor for the definition, the four-region layout, the five states and their diagram, and the seven PCB categories. *Page-verified.*
- **Stallings, *Operating Systems: Internals and Design Principles*, 6e** — Ch. 3 §3.1 (what is a process, p. 108), §3.2 (two-, five-, and seven-state models, p. 111), §3.3 (process image and PCB, p. 126). Use it for the two-state motivation and the three-part PCB grouping. *Page-verified.*
- **Bovet & Cesati, *Understanding the Linux Kernel*, 3e** — Ch. 3 (pp. 79–130): the process descriptor `task_struct` and its `state` field. Use only for the "real PCB" slide. *Page-verified.*
- **Prepare the demo.** Compile a trivial `counter.c` (a loop printing an incrementing global with a short sleep). Run it in two terminals and, in a third, run `ps -o pid,stat,cmd -C counter`: two PIDs, one program. The `STAT` column (`S` sleeping, `R` runnable) is a free preview of states.
- **Draw Figure 5.2 from memory** — five boxes, six arrows, each arrow with its cause. If you cannot name the cause of every arrow, reread p. 103.
- **Prerequisite check.** Students need Lecture 3's blocking `read` and Lecture 2's timer interrupt; both reappear as state-transition causes.

Teaching note: The thesis for the hour: a process is a program in execution, and the PCB is everything the kernel must write down so that execution can be interrupted and resumed. Every section builds one half of that sentence.

5.2 Opening and Program versus Process

Open with the demo or the question: one file, `./counter`, run in two terminals; each copy counts at its own pace. There is one file on disk — so what are the two things the OS is running? The file does not change while it runs; each run has its own counter value, its own position in the code, its own stack; and the OS must remember this for each run separately. Name the runs `P1` and `P2` and keep them on the board all hour. The roadmap is four questions: what is a process; what does it look like in memory; what states can it be in; how does the kernel remember it.

The recipe image sets the contrast: a program is a recipe written down; a process is somebody cooking it, oven on, finger on line 7. Then make it precise: a program is a file on disk, passive, one copy however many runs, existing until deleted; a process is a program in execution, active (a program counter and live resources), one per run, created, run, and terminated. A program becomes a process when its executable is loaded into memory.

Definition (Process). *A process is a program in execution: the unit of work in a modern time-sharing system. It consists of the program code together with its current activity — the program counter, the processor's registers, and its memory.*

Mention that batch systems said *job* and Silberschatz uses the two words almost interchangeably (p. 101), and Stallings' equivalent views (p. 108): a program in execution; an instance of a program running on a computer; the entity that can be assigned to and executed on a processor.

Board: `./counter` twice. *On the board, in order:* (i) draw one disk icon labelled `./counter`; (ii) draw two arrows out of it to two boxes `P1`, `P2`; (iii) in each box write "PC = …", "`count` = …"; (iv) fill in 812 and 305. *Say:* "One recipe, two cooks. `count` has two values at once."

Board: three browser windows. *On the board, in order:* (i) draw three window rectangles; (ii) ask "one process or three?"; (iii) draw both possible answers — one process box owning three windows, and three process boxes; (iv) write the deciding question: "how many PCBs?" *Say:* "Windows are what you see; execution sequences are what the OS tracks."

> **Teaching note:** Students conflate "program" and "process" because everyday speech does ("the Chrome process"). Insist on the counting test: if you can run it twice and get two independent counters, there were two processes. Live question before the definition: "If I delete `./counter` while both copies run, do they stop?" (On UNIX they keep running — the process is not the file.)

5.3 A Process in Memory

A process needs a place to live before the kernel can record it. Its address space has four regions: **text** (code), **data** (globals), **heap** (run-time allocation, grows upward), and **stack** (parameters, return addresses, locals; grows downward), with free space between heap and stack (pp. 101–102).

How to present it. Draw bottom-up, following the coordinate map: the text rectangle first ("address 0, the code"), then data, then heap. Leave a deliberately large dashed gap, then draw the stack at the top. Only now draw the two growth arrows into the gap and say: "heap and stack grow toward each other; the gap is what they share." Add the right-hand notes last.

Stallings calls text, user data, stack, and the control block together the **process image** (p. 126): the first three are what the process uses; the control block is what the kernel uses to manage it. The regions describe what the process is doing; the control block describes what the kernel knows about it.

Board: the deck's fragment. *On the board, in order:* (i) write `int count = 0;` above `main`, then `int step = 1;`, `int *log = malloc(64);`, and the loop `count += step;`; (ii) ask for each

Concepts That Click

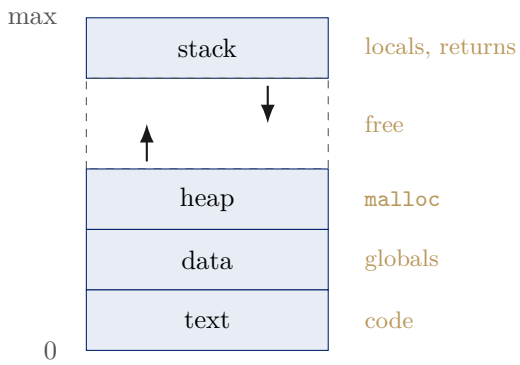


Figure 5.1: The four regions of an address space (Silberschatz et al., Ch. 3 §3.1, pp. 101–102).

line "which region?"; (iii) draw an arrow from each line to the figure: `count`→data, `step`→stack, the 64 bytes→heap, `log` itself→stack, the loop→text. *Say:* "The pointer and what it points to live in different places."

Board: a second instance. *On the board, in order:* (i) write `static char buf[100];` at file scope, a function `f(int x)` with local `double y`, and `char *s = malloc(32)` inside `f`; (ii) ask the class to place `buf`, `x`, `y`, `s`, the 32 bytes, and `f`'s return address; (iii) reveal: data, stack, stack, stack, heap, stack. *Say:* "Parameters and return addresses are on the stack too — that is how a call returns."

> **Teaching note:** The classic error is putting the pointer on the heap with the memory it points to. A second error: thinking two processes share data because they share code. Ask: "If `P1` sets `count = 0`, does `P2`'s counter reset?" The answer (no) is the whole point of per-process data.

5.4 Process States and the State Diagram

A process is not always running; mostly it waits — for the CPU or for the outside world. The five states are new (being created), ready (waiting to be assigned to a processor), running (instructions executing), waiting (for an event such as I/O completion or a signal; Stallings: *blocked*), and terminated (finished; Stallings: *exit*) (Silberschatz p. 103; Stallings p. 111). The counting rule: on one processor at most one process runs at any instant; with k CPUs at most k.

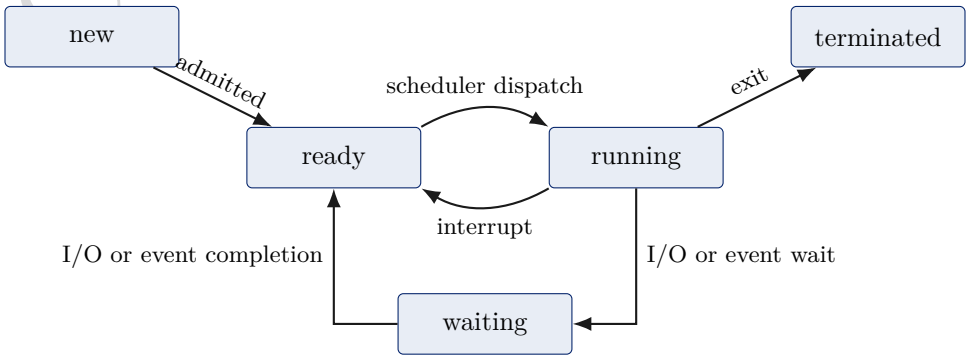


Figure 5.2: The process-state diagram, after Silberschatz et al., Ch. 3 §3.1, p. 103.

Concepts That Click

How to present it. Build it in the order a process lives: draw *ready* and *running* first, side by side, with the dispatch arc above ("the scheduler chooses") and the interrupt arc below ("the timer takes it back"). Then add *waiting* underneath with the two edges, then *new* at top-left and *terminated* at top-right. Label each arrow with its cause as you draw it: admitted; scheduler dispatch; interrupt; I/O or event wait (a blocking system call such as `read`); I/O or event completion; exit (or killed).

Then name the two missing arrows. No waiting→running: completion only makes the process eligible — another process is probably running and only the scheduler chooses; keeping "eligible" and "chosen" separate is what lets the OS have a scheduling policy at all. No ready→waiting: a ready process is not executing, so it cannot request I/O.

Stallings motivates the five states from a **two-state** model (running / not running): too coarse, because a not-running process may be ready or blocked, and dispatching a blocked one wastes the CPU. Splitting *not running* into ready and blocked, and adding new and exit, gives the five-state model; Stallings later adds *suspended* states for swapped-out processes (seven states), which return with the medium-term scheduler.

Board: `P1` reads a key. *On the board, in order:* (i) put a magnet or a circled "P1" on *new*; (ii) move it, narrating: admitted→ready; dispatched→running (increments `count`); timer→ready (`P2` runs); dispatched→running; `read` on keyboard→waiting; key pressed→ready; dispatched→running; `exit`→terminated; (iii) tally: seven states visited, six transitions. *Say:* "The blocking `read` is Lecture 3's system call seen from the process's side."

Board: counting on one CPU. *On the board, in order:* (i) three columns: Running, Ready, Waiting; (ii) write `P1` | `P4`, `P5` | `P2`, `P3`; (iii) announce "`P1`'s slice expires" and ask the class to draw the next state; (iv) reveal: `P4` | `P5`, `P1` | `P2`, `P3`; counts 1, 2, 2; total still five. *Say:* "Transitions move processes; they never create them."

> **Teaching note:** Prediction trap: before revealing the diagram's missing arrows, ask "a process's disk read just finished — does it run now?" Most say yes. That wrong answer is the most useful moment of the hour. Also watch for students who draw a "ready→waiting" arrow "because it is waiting for the CPU" — separate *waiting for the CPU* (ready) from *waiting for an event* (waiting).

5.5 The Process Control Block

When `P1` stops running, everything it was doing must be written down or it can never be resumed.
Definition (Process control block). *The PCB (also called a task control block) is the kernel data structure that represents one process: the repository for any information that may vary from process to process (Silberschatz p. 103).*

One PCB per process — `P1` and `P2` have two, though they share one file. It lives in kernel memory; the process cannot read or write it directly. Creating a process creates a PCB; terminating it eventually frees the PCB.

How to present it. Draw it as a tall stack of cells, top to bottom, and fill each cell by asking "what would you need to resume `P1`?" Students will supply the PC and registers first; prompt for state, PID, memory limits, and open files. Then give the textbook's seven categories (pp. 103–104): process state; program counter; CPU registers (accumulators, index registers, stack pointers, general-purpose registers, condition codes); CPU-scheduling information (priority, queue pointers); memory-management information (base/limit registers, page or segment tables); accounting information (CPU time used, time limits, job or process numbers); I/O status information (devices allocated, open files).

Concepts That Click

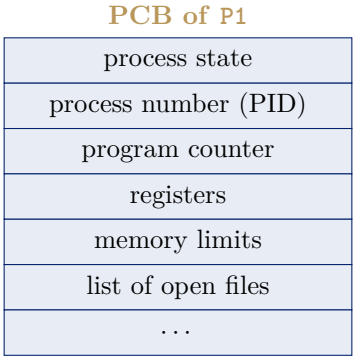


Figure 5.3: PCB fields, after Silberschatz et al., Ch. 3 §3.1, p. 103.

Why the PC and registers? When an interrupt takes the CPU from `P1`, the registers still hold `P1`'s values and `P2` is about to overwrite them; so the kernel copies them into `P1`'s PCB first, and later copies them back, and `P1` continues as if nothing happened. That save-and-load is the **context switch**, next lecture's topic.

Stallings groups the same content into three parts (p. 126): identification (PID, parent's PID, user ID); processor state (user-visible registers, control and status registers such as PC and flags, stack pointers) — exactly what a context switch saves; and process control (scheduling and state, links to other processes, privileges, memory and resource ownership). In Linux the PCB is the process descriptor `task_struct` (Bovet & Cesati, Ch. 3), whose `state` field takes values such as `TASK_RUNNING`, `TASK_INTERRUPTIBLE`, `TASK_UNINTERRUPTIBLE`; all descriptors are linked into a list. Linux uses `TASK_RUNNING` for both ready and running.

Board: two PCBs, one program. *On the board, in order:* (i) draw two PCB stacks side by side; (ii) fill `P1`: waiting (keyboard), PID 4120, PC inside the `read` wrapper, `count` 812, terminal A; (iii) fill `P2`: running, 4127, inside the loop, 305, terminal B; (iv) draw one shared text box underneath both. *Say:* "Every row that differs is why the PCB exists. The PIDs are illustrative."

Board: save and restore. *On the board, in order:* (i) draw a CPU box holding "PC, R1..Rn"; (ii) draw `P1`'s and `P2`'s PCBs; (iii) arrow 1: CPU→PCB(`P1`) labelled "save"; (iv) arrow 2: PCB(`P2`)→CPU labelled "load"; (v) ask "what if step 1 were skipped?" — `P1`'s progress is lost. *Say:* "The PCB makes interruption safe."

> **Teaching note:** Live question: "Why not keep the PCB on the process's own stack, where it is cheaper to reach?" Wait for both answers: protection (the process could rewrite its own state, PID, or memory limits) and availability (the kernel needs it while the process is not running, possibly swapped out). Students often give only the first.

5.6 Threads, Misconceptions, and Close

A process is its address space plus its PCB. One refinement: so far a process has a single thread of execution, one PC. Modern systems allow **multiple threads**, each with its own PC and registers, sharing the address space, and the PCB is extended with per-thread information (pp. 104–105; Ch. 4 in full). A one-thread word processor cannot type and spell-check at the same time; with two threads it can. Keep this to two minutes.

Close with the four misconceptions in wrong/right pairs: a process is not the file but a program in execution; two runs do not share a PCB — one PCB per process; I/O completion does not put

Concepts That Click

a process on the CPU — it goes to ready and the scheduler decides; many processes cannot run on one CPU — at most one, the rest ready or waiting.

5.7 Exit Ticket / Comprehension Checks

1. Three windows of one browser: one process or three? *Answer:* undecidable from windows; count the PCBs.
2. Why does a waiting process go to ready, not running? *Answer:* completion makes it eligible; the scheduler chooses.
3. Why is the PCB not on the process's stack? *Answer:* protection and availability.
4. Five processes: `P1` running, `P2`, `P3` waiting, `P4`, `P5` ready; `P1`'s slice expires and `P4` is dispatched. Counts? *Answer:* running 1, ready 2, waiting 2.

5.8 Summary: One-Page Refresh

- A **process** is a program in execution; program passive, process active.
- Address space: text, data, heap (up), stack (down); two runs share text only.
- Five states; six transitions — admitted, dispatch, interrupt, I/O/event wait, completion, exit; no waiting→running, no ready→waiting. At most one running per CPU.
- The **PCB**: state, PC, registers, scheduling, memory-management, accounting, I/O status — one per process, in kernel memory. Linux: `task_struct`.
- Saving PC and registers into the PCB makes interruption and resumption possible — the context switch, next lecture.

5.9 Readings to Prepare Next Time

- **Silberschatz et al., 8e, Ch. 3 §3.2** — process scheduling and queues (p. 105), context switch (p. 110). *(Page-verified.)*
- **Stallings, 6e, Ch. 3 §3.2 and §3.4** — suspended states (p. 111) and process control / context switch (p. 135). *(Page-verified.)*
- **Bovet & Cesati, 3e, Ch. 3** — the process switch (pp. 79–130). *(Page-verified.)*

Concepts That Click